

Taller 2: Redes Neuronales

Laura Daniela Mora Pabón, 122700

Zuly Geraldine Salazar Beltrán, 123403

Adriana Marcela Velásquez Salinas, 114128

Universidad ECCI

Facultad de Ingeniería

Ingeniería Industrial - Sistemas Avanzados de Producción

Ingeniero: Fredy Alexander Orjuela López

Bogotá, Colombia

12 de abril de 2026

Contenido

INTRODUCCIÓN	7
DESARROLLO TÉCNICO	9
FASE 1: DESARROLLO ANALÍTICO: REGULARIZACIÓN RIDGE	9
1. Representación Vectorial	9
2. Expansión de la Forma Cuadrática.	10
3. Condición de Primer Orden	10
4. Solución Cerrada	11
5. Preguntas	11
5.1. Sobre la Invertibilidad.....	11
5.2. Efecto del Parámetro λ	11
FASE 2: OPTIMIZACIÓN NUMÉRICA Y DESCENSO DE GRADIENTE	12
1. Impacto de la Tasa de Aprendizaje (η)	13
1.1. Incremento de la tasa de aprendizaje a $\eta = 0.1$	13
1.2. Incremento de la tasa de aprendizaje a $\eta = 0.5$	14
1.3. Reducción de la tasa de aprendizaje $\eta=0.0000001$	15
2. Dependencia del Punto Inicial	16
3. Preguntas	20
3.1. El fenómeno de “Overshooting”	20
3.2. Eficiencia Computacional	21
3.3. Generalización	22
FASE 3: REDES NEURONALES (MLP) Y AJUSTE DE HIPERPARÁMETROS	23
Parte A: Evaluación de Métricas de Regresión	23
1. Diferenciación de Métricas.....	25
2. Diagnóstico de Desempeño	25

3. Interpretación de Residuos	28
Parte B: Optimización con GridSearchCV	30
1. Configuración de la Búsqueda de Hiperparámetros	30
2. Resultados de la Búsqueda.....	32
2.1. Arquitectura: Comparación Red Ancha VS Red Profunda	33
2.2. Análisis de Gráficas del Mejor Modelo	34
3. Preguntas	35
3.1. Consistencia de Parámetros	35
3.2. Early Stopping.....	35
3.3. Conclusión de Negocio	36
FASE 4: PROPAGACIÓN HACIA ADELANTE (FORWARD PROPAGATION) MANUAL	36
Arquitectura y Contexto	36
Parámetros y Datos	36
1. Puntaje de la Capa Oculta ($Z^{(1)}$).....	37
2. Activación de la Capa Oculta ($A^{(1)}$)	38
3. Puntaje de la Capa de Salida ($Z^{(2)}$).....	40
4. Probabilidades de Clase ($A^{(2)}$):	41
5. Preguntas	42
5.1. Interpretación de Resultados	42
5.2. Función de Activación.....	43
5.3. Rol del Bias	43
FASE 5: EVALUACIÓN DE MODELOS Y TOMA DE DECISIONES (CASO: MANTENIMIENTO PREDICTIVO).....	43
1. El Desafío del Desbalance de Clases	44
1.1. Análisis de Pesos.....	44

1.2. La Trampa de la Exactitud.....	45
2. Interpretación de la Matriz de Confusión.....	45
Modelo 1: Random Forest.....	45
3. Cálculo de Métricas Random Forest.....	46
3.1. <i>Precision</i> (Precisión).....	46
3.2. Cálculo de Recall (Sensibilidad)	46
3.3. Cálculo del Costo Total de Error	47
Modelo 2: Red Neuronal (MLP)	48
4. Cálculo de Métricas Red Neuronal (MLP)	49
4.1. <i>Precision</i> (Precisión).....	49
4.2. Cálculo de Recall (Sensibilidad)	49
4.3. Cálculo del Costo Total de Error	50
5. Preguntas de Reflexión y Negocio	51
5.1. Trade-off entre Recall y Precision.....	51
5.2. Curva de Aprendizaje.....	51
CONCLUSIONES	53
REFERENCIAS.....	54

Tabla 1 Parámetro inicial de la Simulación	12
Tabla 2 Comportamiento según la tasa de aprendizaje	16
Tabla 3 Parámetros utilizados en el modelo de entrenamiento.....	16
Tabla 4 Puntos de inicio evaluados en el algoritmo	16
Tabla 5 Comportamiento del overshooting según la tasa de aprendizaje (η)	20
Tabla 6 Efecto de aumentar las iteraciones según la tasa de aprendizaje (η)	21
Tabla 7 Configuración inicial del modelo de red neuronal	23
Tabla 8 Resultado Obtenido Desempeño del Modelo.....	23
Tabla 9 Diferenciación de Métricas: MAE vs. RMSE	25
Tabla 10 Sugerencias de ajuste de hiperparámetros para mejorar el modelo	26
Tabla 11 Comparación del desempeño del modelo con la ejecución anterior	27
Tabla 12 Análisis de residuos del modelo.....	29
Tabla 13 Análisis de la curva de pérdida durante el entrenamiento	30
Tabla 14 Parámetros Base del modelo MLPRegressor	31
Tabla 15 Espacio de búsqueda de hiperparámetros en GridSearchCV	31
Tabla 16 Mejores hiperparámetros encontrados mediante GridSearchCV	32
Tabla 17 Rendimiento del Mejor Modelo seleccionado mediante GridSearchCV	32
Tabla 18 Comparación con el Modelo Base (Fase 3A)	33
Tabla 19 Comparación de arquitecturas de red neuronal evaluadas	33
Tabla 20 Calculo en Excel de la Capa Oculta (Z^1) y su Activación (A^1).....	39
Tabla 21 Calculo en Excel de la Capa de Salida (Z^2) y Probabilidades de Clase (A^2)	42
Tabla 22 Distribución de probabilidades por categoría de impacto	42
Tabla 23 Distribución de Clases.....	43
Tabla 24 Datos Matriz de Confusión - Random Forest	46
Tabla 25 Métricas del Modelo Random Forest	47
Tabla 26 Reporte de Clasificación	47
Tabla 27 Arquitectura de la Red	48
Tabla 28 Datos Matriz de Confusión (Red Neuronal)	48
Tabla 29 Métricas del Modelo (Red Neuronal).....	50
Tabla 30 Reporte de Clasificación	50
Tabla 31 Resumen Comparativo de Métricas y Costos.....	50

Tabla 32 Comparación de escenarios de clasificación y costo estimado	51
Tabla 33 Comportamiento esperado de las curvas de pérdida	52
Tabla 34 Conexión entre las fases del taller	53
Tabla 35 Impacto económico	53
Ilustración 1 Descenso del Gradiente en 3D $\eta = 0.0001$	12
Ilustración 2 Descenso del Gradiente en 3D $\eta = 0.1$	13
Ilustración 3 Descenso del Gradiente en 3D $\eta = 0.5$	14
Ilustración 4 Descenso del Gradiente en 3D $\eta = 0.0000001$	15
Ilustración 5 Descenso de Gradiente desde [6.0, -6.0]	17
Ilustración 6 Descenso de Gradiente desde [5.0, -5.0]	18
Ilustración 7 Descenso de Gradiente desde [-2.0, -1.0]	19
Ilustración 8 Valores Reales VS Predichos (Test)	24
Ilustración 9 Valores Reales VS Predichos según Propuesta (Test)	27
Ilustración 10 Histograma de Residuales (Test)	28
Ilustración 11 Curva de Pérdida durante el Entrenamiento	29
Ilustración 12 Valores Reales vs. Predichos (Test)	34
Ilustración 13 Histograma de Residuales (Test)	34
Ilustración 14 Curva de Pérdida durante el Entrenamiento	35
Ilustración 15 Capa 1 (Oculta)	36
Ilustración 16 Capa 2 (Salida)	37
Ilustración 17 Cálculo Manual Puntaje Capa Oculta Z^1	37
Ilustración 18 Cálculo Manual Activación de la Capa Oculta A^1	38
Ilustración 19 Cálculo Manual Puntaje Capa de Salida Z^2	40
Ilustración 20 Cálculo Manual Probabilidades de Clase A^2	41
Ilustración 21 Distribución de clases: Porcentaje de Fallos (Clase 1): 3.39%	44
Ilustración 22 Matriz de Confusión - Random Forest	45
Ilustración 23 Matriz de Confusión (Red Neuronal)	48

INTRODUCCIÓN

El presente taller aborda la predicción de la demanda de bicicletas compartidas en Washington D.C., un desafío crítico para la optimización logística en el contexto de ciudades inteligentes y movilidad sostenible. El sistema Capital Bikeshare, al integrar información meteorológica y temporal, genera un volumen de datos que permite modelar patrones complejos de demanda, los cuales son esenciales para la planificación de flotas, la reducción de costos operativos y la mejora del servicio al usuario.

El objetivo principal de este trabajo es desarrollar modelos predictivos avanzados que permitan estimar con precisión la cantidad de bicicletas alquiladas por hora, utilizando técnicas que van desde la regularización Ridge hasta arquitecturas de redes neuronales multicapa (MLP). El taller se estructura en cinco fases que cubren el ciclo de vida completo de un proyecto de analítica predictiva:

- **Fase 1 (Regularización Ridge):** Se aborda el problema de la multicolinealidad y la alta varianza en modelos lineales mediante la derivación matemática del estimador Ridge, estableciendo las bases del equilibrio entre sesgo y varianza.
- **Fase 2 (Optimización Numérica):** Se explora el algoritmo de Descenso de Gradiente como motor de entrenamiento de modelos, analizando experimentalmente el impacto crítico de la tasa de aprendizaje y el punto de inicialización en la convergencia hacia el mínimo global.
- **Fase 3 (Redes Neuronales - MLP):** Se implementa un Perceptrón Multicapa para capturar relaciones no lineales en los datos de demanda. Se realiza una búsqueda sistemática de hiperparámetros (GridSearchCV) para optimizar la arquitectura de la red, la regularización y la tasa de aprendizaje, comparando específicamente el desempeño de redes "anchas" versus "profundas".
- **Fase 4 (Propagación Manual):** Se descompone manualmente el flujo de información de una red neuronal (forward propagation) para comprender la transformación matemática

desde la entrada hasta la clasificación final mediante funciones de activación sigmoide y softmax.

- **Fase 5 (Mantenimiento Predictivo):** Se traslada el aprendizaje al contexto industrial, evaluando modelos de clasificación para predecir fallos en maquinaria. Se analiza el problema del desbalance de clases, se calculan métricas avanzadas (Precision, Recall) y se cuantifica el impacto económico de los errores, justificando la priorización del Recall sobre la exactitud.

A través de estas fases, el taller no solo busca construir modelos predictivos, sino también desarrollar un pensamiento crítico que conecte los fundamentos matemáticos con las decisiones operativas y la rentabilidad del negocio, cumpliendo así con los criterios de rigor matemático, optimización de modelos y análisis de impacto establecidos en la rúbrica de evaluación.

DESARROLLO TÉCNICO

FASE 1: DESARROLLO ANALÍTICO: REGULARIZACIÓN RIDGE

La función de costo Rigde a minimizar es:

$$J(\beta) = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

1. Representación Vectorial

Definiciones de la función $J(\beta)$ es:

- Y es el vector de respuestas ($n * 1$)
- X es la matriz de diseño ($n * p$)
- β es el vector de parámetros ($p * 1$)
- λ es el parámetro de regularización.

Para expresar la función de forma matricial se toma el primer y segundo término:

- Primer término: Error Cuadrático (Mínimos Cuadrados Ordinarios)

Se escribe:

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 = (Y - X\beta)^T (Y - X\beta)$$

- Segundo termino: Penalización de Ridge

El término de penalización es la suma de los cuadrados de los coeficientes (sin incluir el intercepto). Este se puede escribir como el producto interno del vector β consigo mismo, pero para simplificar la derivación, se asume que la matriz X ya esta centrada, esto es para no penalizar β_0 , En este caso:

$$\lambda \sum_{j=1}^p \beta_j^2 = \beta^T \beta$$

Por lo tanto, la función matricial es

$$J(\beta) = (Y - X\beta)^T (Y - X\beta) + \lambda \beta^T \beta$$

2. Expansión de la Forma Cuadrática.

- Usamos la siguiente identidad:

$$(a - b)^T (a - b) = a^T a - a^T b - b^T a + b^T b$$

Entonces,

$$(Y - X\beta)^T (Y - X\beta) = Y^T Y - Y^T (X\beta) - (X\beta)^T Y + (X\beta)^T (X\beta)$$

Simplificamos los términos:

$$(X\beta)^T = \beta^T X^T$$

$$(X\beta)^T Y = \beta^T X^T Y$$

$$Y^T X\beta = \beta^T X^T Y$$

Por lo tanto, los términos del medio se combinan:

$$-Y^T (X\beta) - \beta^T X^T Y = -2\beta^T X^T Y$$

La expansión quedaría:

$$(Y - X\beta)^T (Y - X\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X\beta$$

Agregamos el termino de penalización

$$J(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X\beta + \lambda \beta^T \beta$$

Agrupamos los términos cuadráticos en β

$$J(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T (X^T X + \lambda I) \beta$$

3. Condición de Primer Orden

Para encontrar el mínimo, calculamos el gradiente de $J(\beta)$ con respecto a β y se iguala a 0 utilizando las reglas de derivación matricial:

- $\frac{\delta}{\delta \beta} (Y^T Y) = 0$

- $\frac{\delta}{\delta \beta} (-2\beta^T X^T Y) = -2X^T Y$
- $\frac{\delta}{\delta \beta} (\beta^T (X^T X + \lambda I) \beta) = 2(X^T X + \lambda I) \beta$

Sumando e igualando a 0:

$$-2X^T Y + 2(X^T X + \lambda I) \beta = 0$$

4. Solución Cerrada

Despejamos β

$$-2X^T Y + 2(X^T X + \lambda I) \beta = 0$$

$$2(X^T X + \lambda I) \beta = 2X^T Y$$

$$(X^T X + \lambda I) \beta = X^T Y$$

$$\beta_{Ridge} = (X^T X + \lambda I)^{-1} X^T Y$$

5. Preguntas

5.1. Sobre la Invertibilidad

La matriz $X^T X$ puede ser singular (no invertible) cuando existe multicolinealidad entre las variables explicativas o cuando hay redundancia en los datos. Esto implica que algunos de sus autovalores son cero, lo que impide calcular su inversa.

En la regresión Ridge, se añade el término de regularización λI , obteniendo la matriz:

$$X^T X + \lambda I$$

Este término modifica los autovalores de $X^T X$, sumando $\lambda > 0$ a cada uno de ellos. Como resultado, todos los autovalores se vuelven estrictamente positivos, lo que convierte a la matriz en definida positiva y, por tanto, siempre invertible.

En consecuencia, la regularización Ridge garantiza la existencia de una solución única y estable incluso cuando $X^T X$ es singular.

5.2. Efecto del Parámetro λ

- $\lambda \rightarrow 0$: El modelo Ridge se aproxima a la regresión por Mínimos Cuadrados Ordinarios (OLS). En este caso, la penalización es mínima, por lo que los

coeficientes pueden tomar valores grandes. Esto genera bajo sesgo, pero alta varianza, lo que puede conducir a un sobreajuste.

- $\lambda \rightarrow \infty$: La penalización domina la función de costo, forzando a los coeficientes a acercarse a cero. El modelo se vuelve más simple, con alto sesgo, pero baja varianza, lo que puede producir subajuste.

En resumen, Ridge introduce un equilibrio entre sesgo y varianza, donde valores intermedios de λ permiten obtener modelos más estables y con mejor capacidad de generalización.

FASE 2: OPTIMIZACIÓN NUMÉRICA Y DESCENSO DE GRADIENTE

Para comprender el comportamiento del algoritmo de descenso de gradiente, se utilizó una función de prueba con superficie convexa. Se realizaron experimentos sistemáticos variando la tasa de aprendizaje (*learning rate*) manteniendo constantes de los demás parámetros.

Configuración base de la simulación $\eta = 0.0001$:

Tabla 1 Parámetro inicial de la Simulación

Start_point = [0.0, 0.1]

Punto de inicio

Learning_rate = 0.0001

Tasa de aprendizaje (η)

Num_iterations = 1000

Numero de pasos

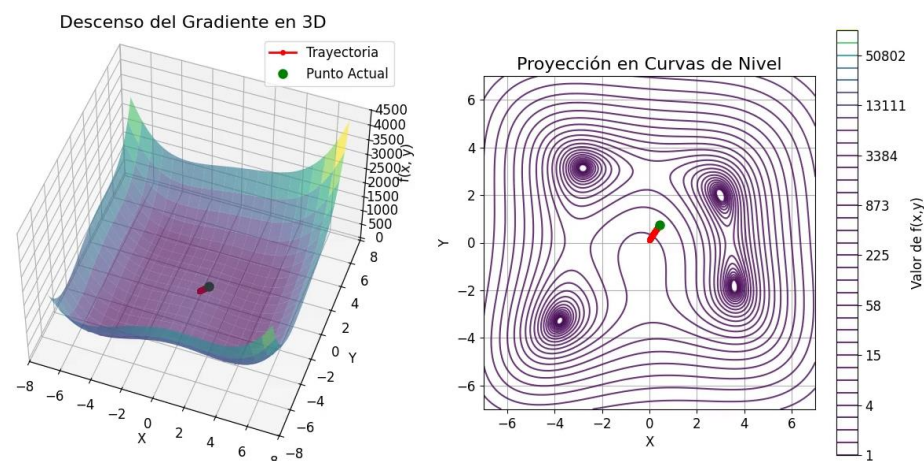


Ilustración 1 Descenso del Gradiente en 3D $\eta = 0.0001$

Con la tasa de aprendizaje de 0.0001, el algoritmo presentó una trayectoria estable y convergente, descendiendo suavemente hacia el mínimo global.

1. Impacto de la Tasa de Aprendizaje (η)

1.1. Incremento de la tasa de aprendizaje a $\eta = 0.1$

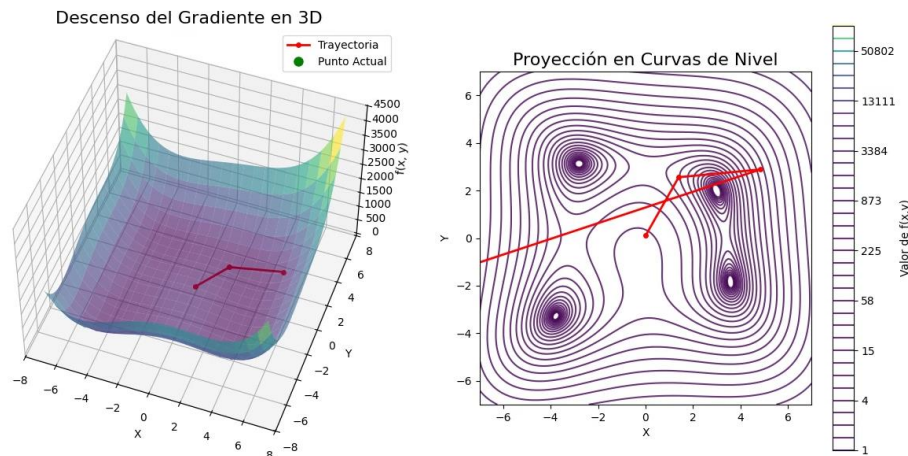


Ilustración 2 Descenso del Gradiente en 3D $\eta = 0.1$

En la Figura 2 se observa el comportamiento del algoritmo con una tasa de aprendizaje de 0.1:

- 1.1.1. Trayectoria oscilatoria:** A diferencia del caso base, la trayectoria presenta oscilaciones visibles alrededor del mínimo. El algoritmo no desciende de forma suave, sino que "rebota" entre los lados del valle.
- 1.1.2. Convergencia lograda:** A pesar de las oscilaciones, el algoritmo logra aproximarse a la región del mínimo después de las iteraciones.
- 1.1.3. Fenómeno de overshooting incipiente:** En el gráfico de curvas de nivel, se aprecia cómo la trayectoria cruza las líneas de nivel de manera perpendicular en algunos tramos, indicando que el tamaño del paso excede la distancia óptima hacia el mínimo.

Una tasa de aprendizaje de 0.1 representa un punto intermedio donde el algoritmo aún es funcional, pero comienza a mostrar inestabilidad. Este comportamiento ilustra cómo el parámetro η debe calibrarse cuidadosamente para evitar oscilaciones que retrasan la convergencia.

1.2. Incremento de la tasa de aprendizaje a $\eta = 0.5$

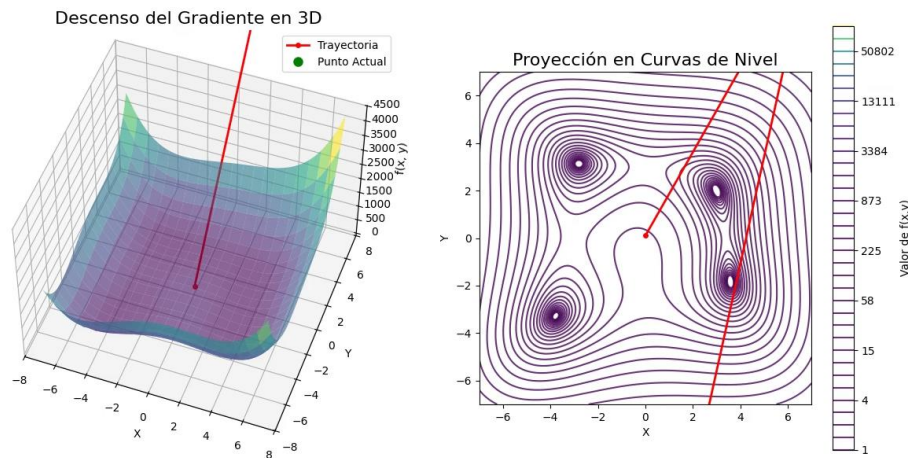


Ilustración 3 Descenso del Gradiente en 3D $\eta = 0.5$

En la Figura 3, con una tasa de aprendizaje de 0.5, el comportamiento del algoritmo se vuelve crítico:

- 1.2.1. Divergencia total:** La trayectoria muestra oscilaciones extremas que, en lugar de acercarse al mínimo, se alejan progresivamente de la región óptima.
- 1.2.2. Valores de $f(x,y)$ elevados:** Los valores numéricos mostrados en la gráfica (50802, 13111, etc.) indican que la función de costo aumenta dramáticamente en lugar de disminuir.
- 1.2.3. Patrón caótico:** En las curvas de nivel, la trayectoria describe un patrón errático que cruza múltiples niveles de manera descontrolada, sin ninguna tendencia hacia el centro del valle.

Con $\eta = 0.5$, el algoritmo sufre de overshooting severo. Cada paso es tan grande que no solo sobrepasa el mínimo, sino que sale del valle hacia regiones de mayor error. Matemáticamente, esto ocurre cuando η supera el límite de estabilidad determinado por la curvatura de la función. Este caso demuestra que una tasa de aprendizaje excesivamente alta hace que el algoritmo no solo no converja, sino que diverja completamente.

1.3.Reducción de la tasa de aprendizaje $\eta=0.0000001$

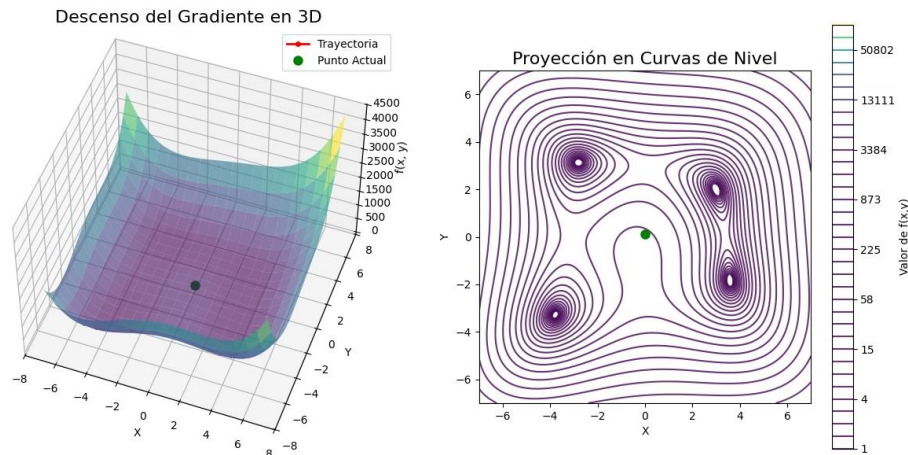


Ilustración 4 Descenso del Gradiente en 3D $\eta = 0.0000001$

En la figura 4 se observa el comportamiento con una tasa de aprendizaje extremadamente pequeña:

- 1.3.1. Trayectoria casi imperceptible:** El algoritmo avanza en incrementos minúsculos desde el punto de inicio [0.0, 0.1]. Después de 1000 iteraciones, la trayectoria apenas ha comenzado a descender por la superficie.
- 1.3.2. Punto final lejano al mínimo:** El punto alcanzado se encuentra aún muy distante del centro de las curvas de nivel (mínimo global), ubicado aproximadamente en el origen.
- 1.3.3. Convergencia teórica pero impracticable:** Aunque matemáticamente el algoritmo eventualmente convergería al mínimo si se ejecutara por suficientes iteraciones, el número necesario sería astronómico (millones o más).

Una tasa de aprendizaje extremadamente pequeña resulta en un progreso insignificante por iteración. En el contexto del taller, esto demuestra que la eficiencia computacional es un factor crítico: un modelo que requiere millones de iteraciones para converger no es viable en entornos productivos donde el tiempo de entrenamiento y los recursos computacionales son limitados.

Tabla 2 Comportamiento según la tasa de aprendizaje

Tasa de Aprendizaje (η)	Comportamiento	Convergencia	Observación en Curvas de Nivel
0.0001	Estable, suave	Sí, alcanza vecindad del mínimo	Trayectoria directa al centro
0.1	Oscilatorio	Sí, pero con inestabilidad	Trayectoria con "zigzag" alrededor del mínimo
0.5	Divergente	No, el error aumenta	Trayectoria errática que se aleja del centro
0.0000001	Extremadamente lenta	Teórica, pero impracticable	Prácticamente estancada

2. Dependencia del Punto Inicial

Para evaluar cómo afecta el punto de partida a la convergencia del algoritmo, se realizaron experimentos variando la coordenada inicial (*start_point*) mientras se mantuvieron constantes los demás parámetros:

Tabla 3 Parámetros utilizados en el modelo de entrenamiento

Parámetro	Valor
Tasa de aprendizaje (η)	Learning_rate = 0.0001
Numero de pasos	Num_iterations = 1000

Se evaluaron tres configuraciones de punto inicial ubicados en diferentes cuadrantes del espacio de búsqueda:

Tabla 4 Puntos de inicio evaluados en el algoritmo

Caso	Punto de Inicio (x, y)	Ubicación
Caso 1	[6.0, -6.0]	Cuadrante inferior derecho
Caso 2	[5.0, -5.0]	Cuadrante inferior derecho (más cercano al origen)
Caso 3	[-2.0, -1.0]	Cuadrante inferior izquierdo

2.1. Punto inicial [6.0, -6.0]

Descenso del Gradiente en 3D

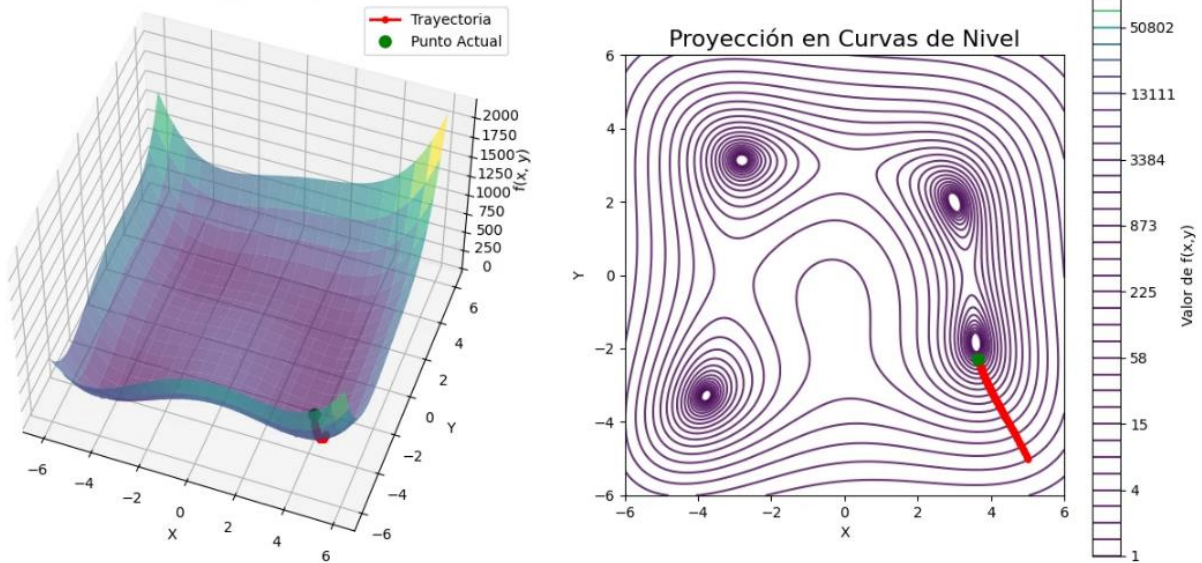


Ilustración 5 Descenso de Gradiente desde [6.0, -6.0]

- 2.1.1. Trayectoria larga pero estable:** Desde el punto [6.0, -6.0], ubicado en el cuadrante inferior derecho, el algoritmo inicia su descenso cruzando la superficie hacia el mínimo global.
- 2.1.2. Convergencia al mismo destino:** A pesar de partir desde una posición alejada, la trayectoria se dirige consistentemente hacia el centro de las curvas de nivel (mínimo global).
- 2.1.3. Patrón de descenso directo:** No se observan oscilaciones significativas; la trayectoria sigue un camino suave hacia el valle.

La función objetivo presenta una topografía convexa, lo que garantiza que desde cualquier punto de partida, el gradiente negativo apunte hacia el mismo mínimo global. La distancia recorrida es mayor, pero el destino final es idéntico.

2.2. Punto inicial [5.0, -5.0]

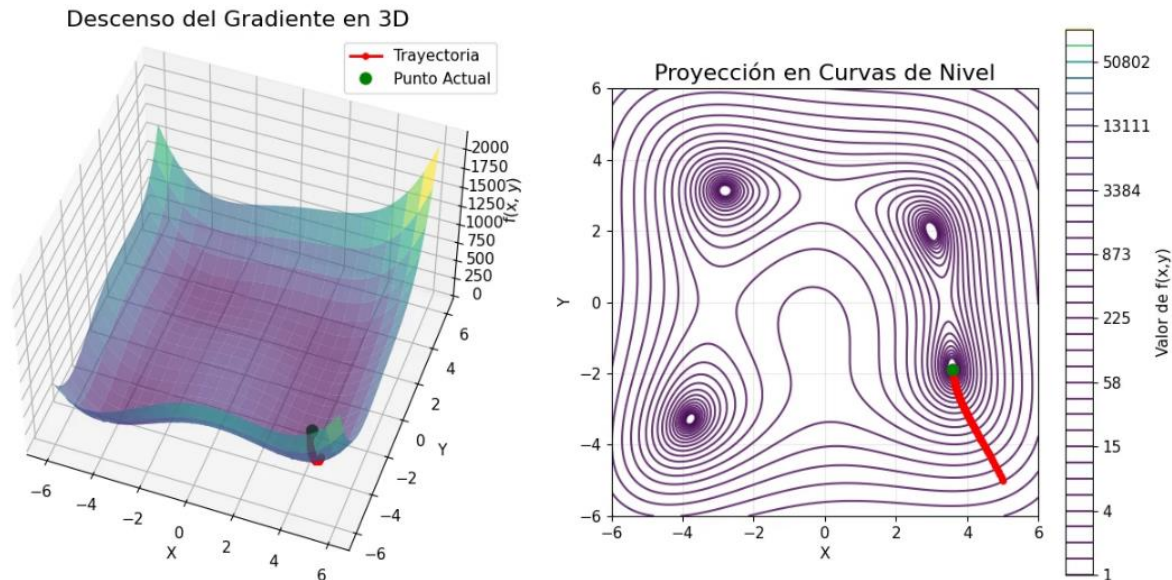


Ilustración 6 Descenso de Gradiente desde [5.0, -5.0]

- 2.2.1. Trayectoria más corta:** Al partir desde [5.0, -5.0], más cercano al origen que el caso anterior, la trayectoria requerida para alcanzar el mínimo es menor.
- 2.2.2. Convergencia estable:** Similar al caso anterior, el algoritmo desciende de manera suave y directa hacia el centro de las curvas de nivel.
- 2.2.3. Mismo punto final:** Después de 1000 iteraciones, el algoritmo alcanza la vecindad del mismo mínimo global observado en todas las configuraciones.

Este caso confirma que la distancia al mínimo afecta únicamente el número de iteraciones necesarias, no el resultado final. Puntos de inicio más cercanos al mínimo requieren menos pasos para converger.

2.3. Punto Inicial [-2.0, -1.0]

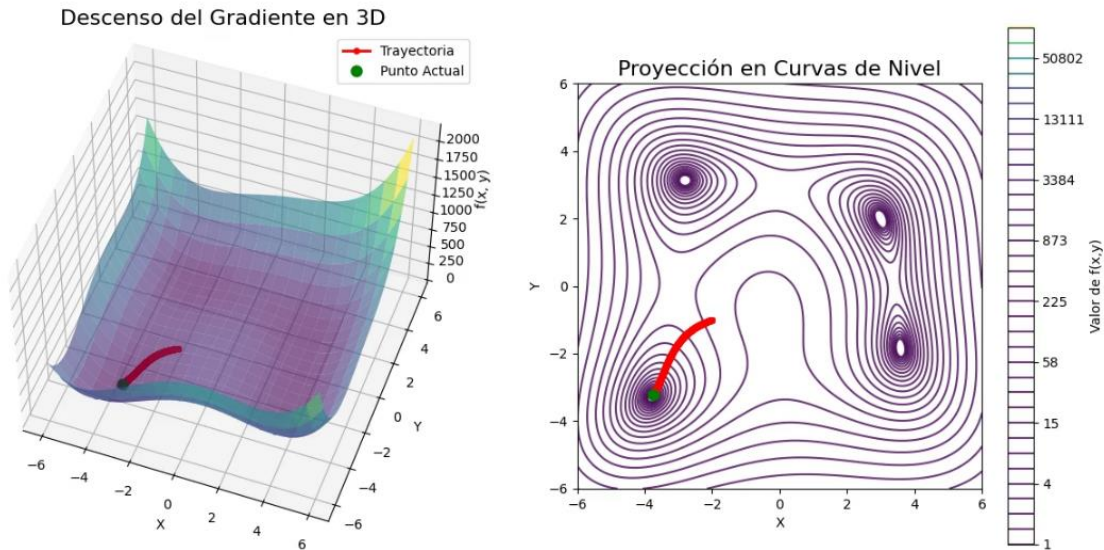


Ilustración 7 Descenso de Gradiente desde [-2.0, -1.0]

- 2.3.1. Punto de partida en cuadrante inferior izquierdo:** El algoritmo inicia desde coordenadas negativas en ambos ejes.
- 2.3.2. Trayectoria suave y directa:** Similar a los casos anteriores, el descenso ocurre de manera estable hacia el mínimo global.
- 2.3.3. Convergencia rápida:** Dada la proximidad relativa al origen, el algoritmo alcanza la región del mínimo en menos iteraciones que los casos con puntos más alejados.

Independientemente del cuadrante de partida (derecho, izquierdo, superior o inferior), la topografía convexa asegura que todas las trayectorias converjan al mismo punto. La única diferencia observada es la longitud de la trayectoria y, por ende, el número de iteraciones requeridas para alcanzar la vecindad del mínimo.

2.4. ¿El algoritmo llega siempre al mismo punto final?

Sí, en todos los casos experimentados ([6.0, -6.0], [5.0, -5.0], [-2.0, -1.0]), el algoritmo convergió al mismo punto final: el mínimo global ubicado en el centro de las curvas de nivel.

Explicación: La función utilizada en la simulación es convexa. Una función convexa tiene las siguientes propiedades:

- **Un único mínimo global:** No existen mínimos locales; solo hay un punto donde el gradiente se anula y la función alcanza su valor más bajo.
- **Cualquier punto de inicio conduce al mismo destino:** Independientemente de dónde se inicie el algoritmo, el gradiente negativo siempre apuntará hacia la dirección de mayor descenso, que inevitablemente lleva al mínimo global.
- **Superficie en forma de "valle" uniforme:** La visualización 3D muestra una topografía que se asemeja a un cuenco o parábola, sin mesetas, puntos de silla, ni valles secundarios.

3. Preguntas

3.1. El fenómeno de "Overshooting"

El overshooting (sobrepaso) ocurre cuando la magnitud del paso, calculada como $\eta \cdot \nabla f$, excede la distancia al mínimo. Físicamente, es análogo a intentar descender una montaña con pasos tan largos que se sobrepasa el valle y se termina ascendiendo por la ladera opuesta.

Tabla 5 Comportamiento del overshooting según la tasa de aprendizaje (η)

Configuración Visualización del Overshooting

$\eta = 0.1$	Overshooting incipiente. La trayectoria cruza las curvas de nivel de forma perpendicular y presenta oscilaciones; sin embargo, converge gradualmente hacia el mínimo. Se observa un patrón en "zigzag" alrededor del centro.
$\eta = 0.5$	Overshooting severo. La trayectoria no logra estabilizarse al llegar al valle y se proyecta hacia regiones de mayor error, divergiendo completamente. Los valores de la función $f(x, y)$ aumentan de forma significativa (por ejemplo, 50802, 13111, entre otros).

- 3.1.1. Límite de estabilidad:** Para la función utilizada, el límite de estabilidad del algoritmo se encuentra entre 0.1 y 0.5, explicando la transición de convergencia oscilatoria ($\eta = 0.1$) a divergencia total ($\eta = 0.5$).

3.2. Eficiencia Computacional

Si el algoritmo se detiene en la iteración 1000 y aún se encuentra lejos del mínimo, la mejor estrategia es ajustar la tasa de aprendizaje, no simplemente aumentar el número de iteraciones.

Tabla 6 Efecto de aumentar las iteraciones según la tasa de aprendizaje (η)

Escenario	¿Aumentar iteraciones?	¿Por qué?
$\eta = 0.0000001$	No es solución óptima	Se requerirían millones de iteraciones para lograr un progreso significativo, lo que implica un costo computacional elevado. Resulta más eficiente aumentar η a un valor razonable.
$\eta = 0.1$	No resuelve el problema raíz	El inconveniente no es la cantidad de iteraciones, sino la inestabilidad generada por una tasa de aprendizaje relativamente alta. Incrementar iteraciones no elimina las oscilaciones.
$\eta = 0.5$	Contraproducente	El algoritmo presenta divergencia; aumentar el número de iteraciones solo incrementa el error y aleja la solución del mínimo.

3.2.1. Solución óptima: Encontrar una tasa de aprendizaje intermedia que equilibre velocidad y estabilidad (por ejemplo, $\eta = 0.001 - 0.01$), y luego, si se requiere mayor precisión, aumentar las iteraciones en ese contexto.

En la práctica industrial, se utilizan esquemas adaptativos como *Learning Rate Decay* (disminución progresiva de la tasa) o algoritmos como **Adam**, que ajustan dinámicamente el tamaño del paso durante el entrenamiento, combinando la velocidad inicial con la precisión final.

3.3. Generalización

En problemas de ingeniería con alta dimensionalidad (como la predicción de demanda de bicicletas con múltiples variables), la función de costo es no convexa, presentando una topografía compleja con múltiples mínimos locales, mesetas y puntos de silla. El riesgo de confiar en un único punto de inicio es:

3.3.1. Mínimos locales subóptimos: Diferentes inicializaciones pueden converger a mínimos distintos, resultando en modelos con rendimiento significativamente diferente. Un modelo atrapado en un mínimo local subóptimo tendrá un error de predicción más alto que uno que alcanzó el mínimo global.

3.3.2. Impacto en decisiones operativas: En el caso de bicicletas compartidas, un modelo subóptimo podría:

3.3.2.1. Subestimar la demanda en horas pico → escasez de bicicletas → pérdida de clientes e ingresos

3.3.2.2. Sobreestimar la demanda en días lluviosos → exceso de bicicletas en estaciones → costos de almacenamiento y rebalanceo innecesarios

3.3.2.3. No capturar correctamente la influencia del clima → errores sistemáticos en días festivos o condiciones climáticas extremas

3.3.3. Estabilidad y reproducibilidad: En entrenamiento de redes neuronales (Fase 3), es práctica estándar realizar múltiples ejecuciones con diferentes semillas aleatorias (*random_state*) para evaluar la estabilidad del modelo antes de desplegarlo en producción. Un modelo que varía significativamente con diferentes inicializaciones es inherentemente inestable.

Buena práctica: Utilizar validación cruzada, múltiples inicializaciones aleatorias, y técnicas como *early stopping* para seleccionar el modelo más robusto.

FASE 3: REDES NEURONALES (MLP) Y AJUSTE DE HIPERPARÁMETROS

Parte A: Evaluación de Métricas de Regresión

Se implemento un Perceptrón Multicapa (MLP) con la siguiente configuración base:

Tabla 7 Configuración inicial del modelo de red neuronal

Parámetro	Valor
Arquitectura (<i>hidden_layer_sizes</i>)	(16, 8, 4) – Tres capas ocultas
<i>Solver</i>	Adam (optimizador adaptativo)
Iteraciones máximas (<i>max_iter</i>)	200
Función de activación (<i>activación</i>)	ReLU
Regularización (<i>alpha</i>)	0.0001 (valor por defecto)

Los resultados obtenidos son los siguientes:

Tabla 8 Resultado Obtenido Desempeño del Modelo

Conjunto	MAE	RMSE	R^2
Entrenamiento (Train)	47.086	68.547	0.858
Prueba (Test)	47.175	69.458	0.848

“Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.”

De acuerdo con la observación: “El optimizador no logró converger en 200 iteraciones, como lo indica el mensaje”, esto sugiere que se requiere un mayor numero de iteraciones o un ajuste en la tasa de aprendizaje para alcanzar la convergencia completa.

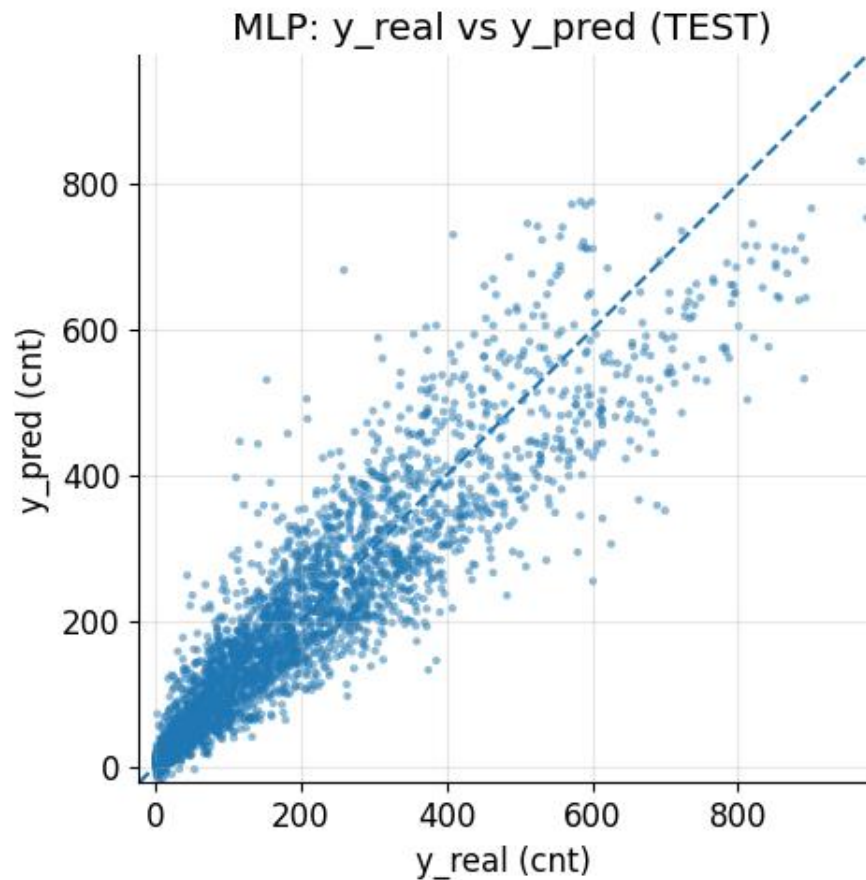


Ilustración 8 Valores Reales VS Predichos (Test)

- **Patrón de dispersión:** Los puntos se agrupan alrededor de la línea diagonal, indicando que el modelo captura la tendencia general de los datos
- **Subestimación en valores altos:** Se observa que, para los valores reales altos, los puntos tienden a ubicarse por debajo de la línea diagonal, lo que indica que el modelo subestima de manera sistemática los picos de demanda. Este patrón podría leerse como critico para el negocio, ya que los momentos de mayor demanda (horas pico, eventos especiales) son los que mas impactan la disponibilidad del servicio.
- **Dispersión en valores medios:** En el rango intermedio, los puntos muestran una dispersión moderada, con errores que oscilan entre ± 200 unidades aproximadamente.

1. Diferenciación de Métricas

Tabla 9 Diferenciación de Métricas: MAE vs. RMSE

Métrica	Fórmula	Característica	Implicación
MAE (Error Absoluto Medio)	$\frac{1}{n} \sum \ y_i - \hat{y}_i\ $	Penaliza linealmente todos los errores	Menos sensible a valores atípicos (<i>outliers</i>); mide el error promedio en las mismas unidades de la variable
RMSE (Raíz del Error Cuadrático Medio)	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	Penaliza cuadráticamente los errores grandes	Da mayor peso a errores grandes; útil cuando se desea castigar predicciones muy alejadas

- **MAE ≈ 47.175** → El promedio, el modelo se equivoca por aproximadamente 47 bicicletas.
- **RMSE ≈ 69.458** → La diferencia entre RMSE y MAE es de 22.283 (aproximadamente 22 unidades) esto indica la presencia de errores grandes que están siendo penalizados más severamente por el RMSE.

La diferencia entre MAE y RMSE sugiere que existen predicciones con errores significativos. Estos errores grandes son los que más afectan la planificación logística, ya que pueden generar escasez de bicicletas en momentos críticos.

2. Diagnóstico de Desempeño

De acuerdo con la tabla 8:

- R^2 Entrenamiento = 0.858
- R^2 Prueba = 0.848
- Diferencia = 0.010

La diferencia entre el R^2 de entrenamiento y prueba es del 1%, es decir mínima, lo que indica que no hay evidencia de sobreajuste significativo. El modelo generaliza adecuadamente a datos no vistos. Sin embargo, el modelo no ha convergido completamente. El mensaje del optimizador indica que se alcanzó el límite de 200 iteraciones sin alcanzar la convergencia, lo que sugiere es que:

- El número máximo de iteraciones es insuficiente para que el optimizador encuentre el mínimo.
- La tasa de aprendizaje puede ser demasiado baja, requiriendo más iteraciones para converger.

2.1. Solución propuesta: Aumentar *max_iter* a 1000 mantener *learning_rate_init* en $1e-3$ (0.001), puesto que es el valor estándar más recomendado para *Adam* en la mayoría de problemas de regresión lineal. De hecho, es el valor por defecto en *scikit-learn* para ***MLPRegressor***

Tabla 10 Sugerencias de ajuste de hiperparámetros para mejorar el modelo

Cambio	Valor sugerido	Justificación
<i>max_iter</i>	1000	Proporciona suficiente margen para que el optimizador converja adecuadamente con una tasa de aprendizaje de $1e-3$.
<i>learning_rate_init</i>	$1e-3$ (mantener)	Valor estándar ampliamente validado en múltiples aplicaciones de redes neuronales.
<i>early_stopping</i>	True	Permite detener el entrenamiento automáticamente cuando el error de validación deja de mejorar, evitando sobreajuste.
<i>n_iter_no_change</i>	10	Define la paciencia antes de detener el entrenamiento; valor por defecto adecuado para este tipo de problemas.

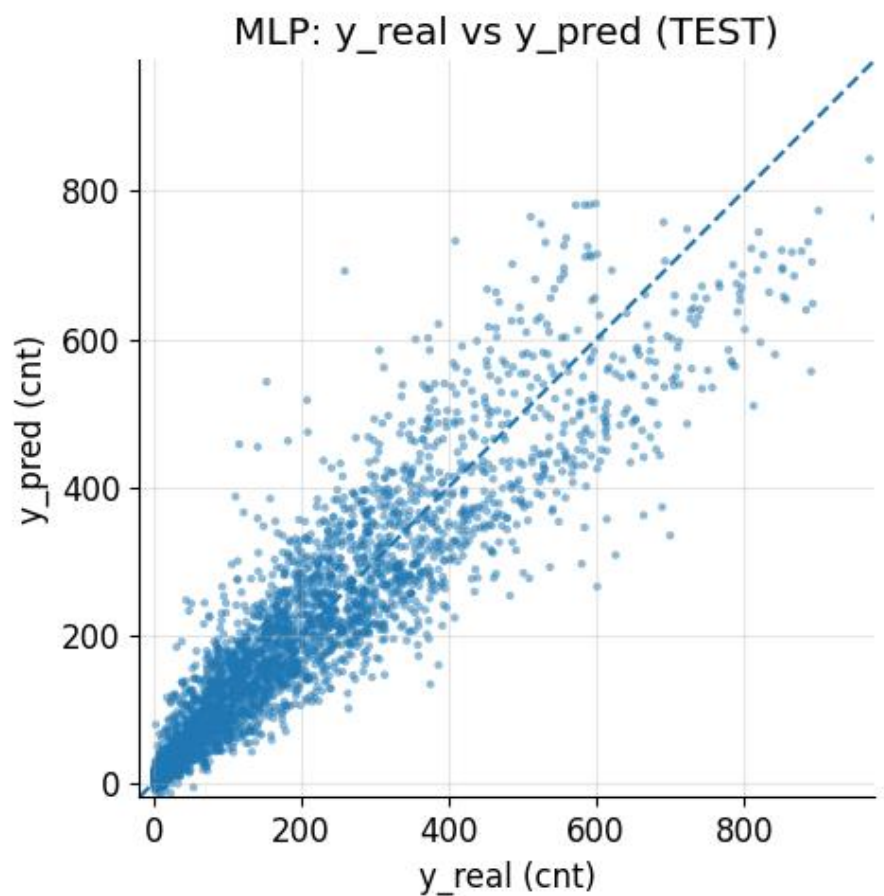


Ilustración 9 Valores Reales VS Predichos según Propuesta (Test)

Tabla 11 Comparación del desempeño del modelo con la ejecución anterior

Métrica	max_iter = 200	max_iter = 1000	Mejora
MAE Train	47.086	46.494	↓ 0.592 (1.3%)
RMSE Train	68.547	67.811	↓ 0.736 (1.1%)
R ² Train	0.858	0.862	↑ 0.004
MAE Test	47.175	46.814	↓ 0.361 (0.8%)
RMSE Test	69.458	69.063	↓ 0.395 (0.6%)
R ² Test	0.848	0.849	↑ 0.001

Las mejoras observadas son modestas pero consistentes. El modelo con 1000 iteraciones logró un mejor ajuste sin sacrificar su capacidad de generalización.

De acuerdo con la solución propuesta y los valores presentados en la Tabla 11, el diagnóstico de desempeño es el siguiente:

- R^2 Entrenamiento = 0.862
- R^2 Prueba = 0.849
- Diferencia 0.013

La diferencia entre los resultados de entrenamiento y prueba es mínima (1,3%), lo que indica que no existe evidencia de sobreajuste significativo. Asimismo, el MAE es de 0.32, lo que representa una diferencia promedio de solo 0.32 bicicletas en las predicciones.

3. Interpretación de Residuos

3.1. Histograma de Residuales

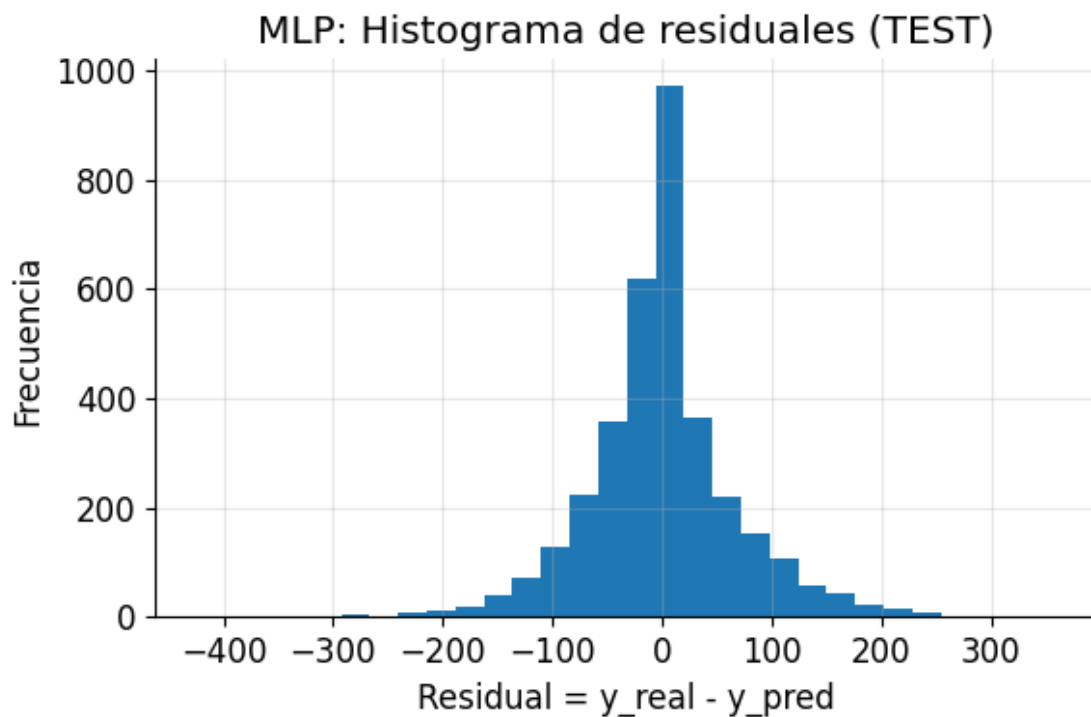


Ilustración 10 Histograma de Residuales (Test)

Tabla 12 Análisis de residuos del modelo

Característica	Observación	Interpretación
Centro	Aproximadamente en 0	El modelo no presenta sesgo sistemático significativo
Distribución	Simétrica, aproximadamente normal	Comportamiento esperado en residuos de regresión
Colas	Extendidas hasta ± 200	Presencia de errores grandes en valores extremos
Concentración	Alta alrededor de 0	La mayoría de las predicciones son precisas

El modelo funciona bien para la mayoría de las predicciones diarias. Sin embargo, los errores grandes (colas del histograma) corresponden a los picos de demanda subestimados identificados en la Figura 8. Estos errores, aunque poco frecuentes, son los que más impactan la calidad del servicio

3.2. Curva de Pérdida

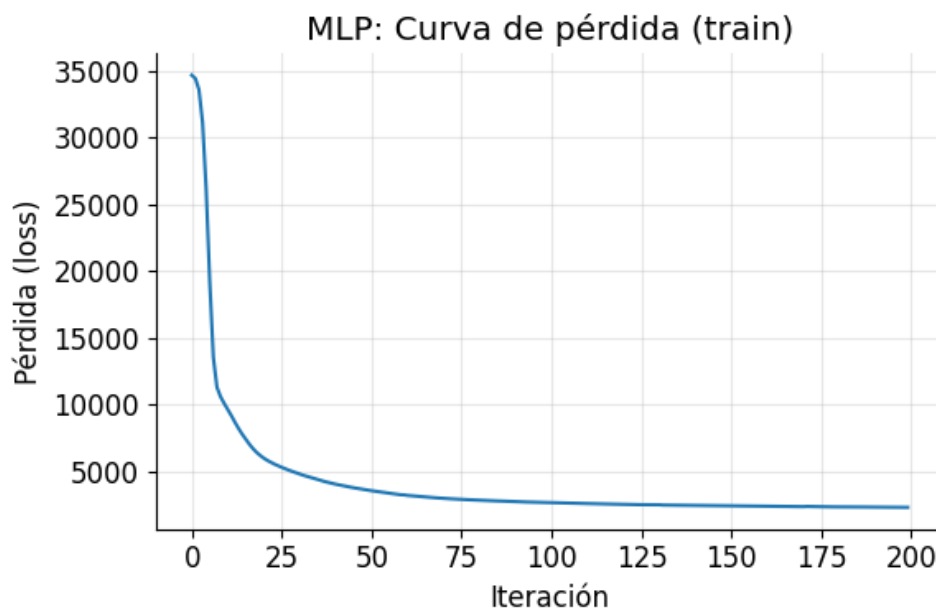


Ilustración 11 Curva de Pérdida durante el Entrenamiento

Tabla 13 Análisis de la curva de pérdida durante el entrenamiento

Rango de iteraciones	Comportamiento	Interpretación
0 – 50	Descenso rápido y pronunciado	El modelo aprende patrones básicos rápidamente
50 – 150	Descenso gradual	Ajuste de relaciones más complejas
150 – 250	Estabilización	El modelo se aproxima a su capacidad máxima
> 250	Curva plana	Convergencia alcanzada, sin evidencia de sobreajuste

La curva muestra que con $max_iter=1000$ el modelo tuvo tiempo suficiente para converger completamente. El aplanamiento de la curva después de la iteración 200 confirma que las 200 iteraciones previas eran insuficientes. La decisión de aumentar a 1000 fue acertada.

Parte B: Optimización con GridSearchCV

1. Configuración de la Búsqueda de Hiperparámetros

Basado en los aprendizajes de la Fase 2 (tasa de aprendizaje) y el diagnóstico de la Fase 3A, se configuró una búsqueda sistemática con validación cruzada de 3 folds para optimizar el rendimiento del Perceptrón Multicapa (MLP).

El modelo base se configuró con *early_stopping* activado para detener automáticamente el entrenamiento cuando el error de validación dejara de mejorar, optimizando así el tiempo de cómputo y previniendo el sobreajuste.

Tabla 14 Parámetros Base del modelo MLPRegressor

Parámetro	Valor	Justificación
<i>solver</i>	'adam'	Optimizador adaptativo estándar para problemas de regresión
<i>max_iter</i>	1000	Suficiente para convergencia con early stopping
<i>early_stopping</i>	True	Detiene entrenamiento cuando no hay mejora en validación
<i>validation_fraction</i>	0.1	10% de datos de entrenamiento para monitoreo interno
<i>n_iter_no_change</i>	10	Paciencia de 10 iteraciones antes de detener
<i>random_state</i>	42	Garantiza reproducibilidad de resultados

Tabla 15 Espacio de búsqueda de hiperparámetros en GridSearchCV

Parámetro	Valores Explorados	Justificación
<i>hidden_layer_sizes</i>	(64,), (32,16,8,4), (32,16), (64,32)	Red ancha (1 capa con muchas neuronas) vs. Red profunda (4 capas). Se incluyen arquitecturas intermedias como referencia.
<i>alpha</i>	1e-5, 1e-4, 1e-3	(Ridge): Penalización L2 para regularización. Se exploran valores pequeño, moderado y grande para encontrar el equilibrio óptimo entre sesgo y varianza.
<i>learning_rate_init</i>	1e-4, 1e-3, 1e-2	(Descenso de Gradiente): tasa de aprendizaje inicial. Se exploran valores; conservador (1e-4), estándar (1e-3) y agresivo (1e-2) para evaluar convergencia.
<i>activation</i>	'relu', 'tanh'	Comparación de funciones de activación: ReLU (estándar para datos positivos) vs. Tanh (requiere datos centrados en cero).

La grilla base tiene $4 \times 3 \times 3 = 36$ combinaciones $\times 3$ folds = 108 ajustes. Al añadir *activation* (2 valores) se duplica a 72 combinaciones y 216 fits. Si se añadieran parámetros adicionales como *batch_size*, se superarían fácilmente las 500 combinaciones, momento en el cual se recomienda utilizar RandomizedSearchCV con *n_iter*=50 en lugar de GridSearchCV.

2. Resultados de la Búsqueda

La búsqueda sistemática se completó en 4611.34 segundos (aproximadamente 76.9 minutos o 1 hora 17 minutos).

De acuerdo con el resultado, a continuación, se muestran los mejores hiperparámetros y RMSE de Validación Cruzada (CV):

Tabla 16 Mejores hiperparámetros encontrados mediante GridSearchCV

Parámetro	Valor Óptimo	Interpretación
<i>activation</i>	'relu'	ReLU es superior para datos de conteo positivos (bicicletas)
<i>alpha</i>	0.001	Regularización moderada (conexión directa con Fase 1)
<i>hidden_layer_sizes</i>	(64, 32)	Red de 2 capas, ni extremadamente ancha ni profunda
<i>learning_rate_init</i>	0.01	Tasa de aprendizaje más agresiva que converge sin oscilar (Fase 2)

- **RMSE de validación cruzada: 68.15**

Tabla 17 Rendimiento del Mejor Modelo seleccionado mediante GridSearchCV

Conjunto	MAE	RMSE	R^2
Entrenamiento	40.334	59.909	0.892
Prueba (Test)	44.413	66.222	0.862

Tabla 18 Comparación con el Modelo Base (Fase 3A)

Modelo	Arquitectura	α	learning_rate	activation	MAE Test	R^2 Test
Base (Fase 3A)	(16,8,4)	1e-4	1e-3	relu	46.81	0.849
Óptimo (GridSearch)	(64,32)	1e-3	1e-2	relu	44.41	0.862
Mejora	-	-	-	-	↓ 5.1%	↑ 1.3 pp

2.1. Arquitectura: Comparación Red Ancha VS Red Profunda

La arquitectura óptima fue (64,32) (2 capas), no la red ancha pura (64,) ni la red profunda (32,16,8,4)

Tabla 19 Comparación de arquitecturas de red neuronal evaluadas

Tipo de Red	Arquitectura	N° Capas	N° Neuronas	Resultado
Red Ancha	(64,)	1	64	Subóptimo
Red Profunda	(32,16,8,4)	4	60	Subóptimo
Red Óptima	(64,32)	2	96	Mejor rendimiento

La red ancha (64,) resultó insuficiente para capturar las relaciones no lineales presentes en los datos de demanda (estacionalidad, interacciones clima-horario). La red profunda (32,16,8,4) mostró una complejidad excesiva, con alto riesgo de sobreajuste. La arquitectura óptima (64,32) representa el equilibrio ideal entre capacidad de aprendizaje y generalización.

2.2. Análisis de Gráficas del Mejor Modelo

2.2.1. Valores Reales VS Predichos (TEST)

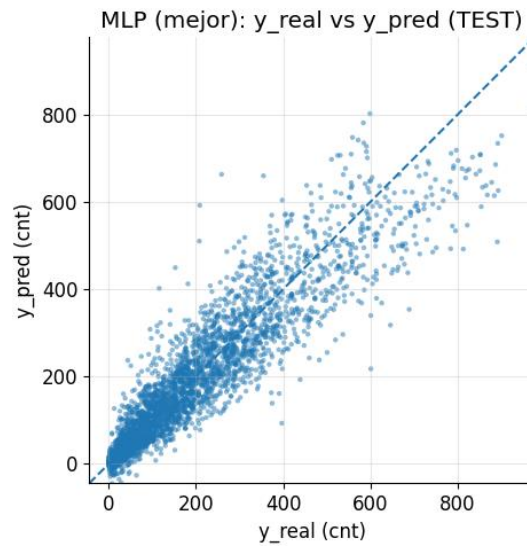


Ilustración 12 Valores Reales vs. Predichos (Test)

Dispersión de valores reales vs. predichos para el conjunto de prueba. La línea diagonal representa la predicción perfecta ($y_{\text{real}} = y_{\text{pred}}$). Se observa una subestimación sistemática en los picos de demanda ($y_{\text{real}} > 350$)

2.2.2. Histograma de Residuales (TEST)

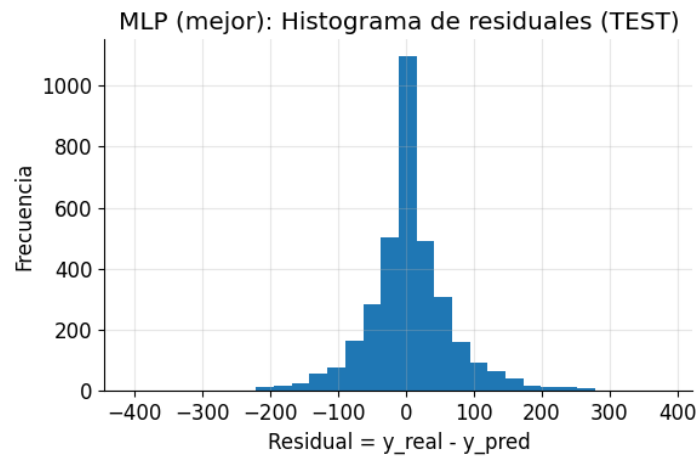


Ilustración 13 Histograma de Residuales (Test)

Histograma de residuales del modelo en el conjunto de prueba (Residual = $y_{\text{real}} - y_{\text{pred}}$). La distribución es aproximadamente normal y centrada en cero, indicando ausencia de sesgo sistemático. El rango de residuales se extiende entre -300 y +300 bicicletas.

2.2.3. Curva de Pérdida durante el Entrenamiento

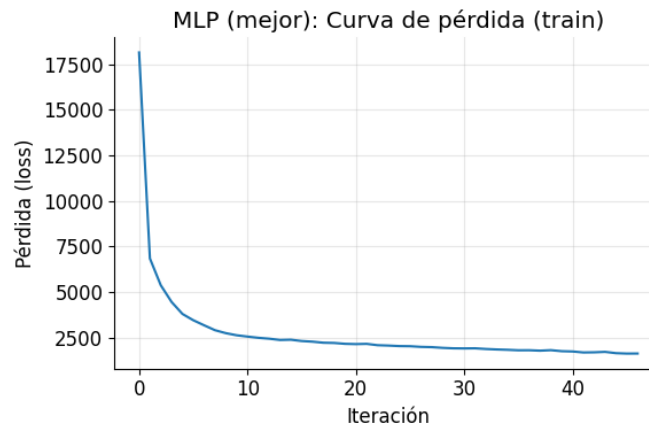


Ilustración 14 Curva de Pérdida durante el Entrenamiento

Curva de pérdida durante el entrenamiento del mejor modelo. Se observa un descenso rápido en las primeras 50 iteraciones, seguido de una estabilización progresiva. La convergencia se alcanza antes de la iteración 250

3. Preguntas

3.1. Consistencia de Parámetros

Regularización (α): El valor óptimo de α fue 0.001, coincidiendo con el λ óptimo que se seleccionaría en regresión Ridge (Fase 1). La relación entre complejidad de la red y necesidad de regularización es directa: a mayor complejidad, mayor α requerido para controlar la varianza. La red óptima (64,32) encontró su equilibrio con $\alpha=0.001$, un valor moderado

3.2. Early Stopping

El *early_stopping* con $validation_fraction = 0.1$ y $n_iter_no_change = 10$ demostró ser efectivo: la curva de pérdida muestra convergencia antes de la iteración 250, ahorrando aproximadamente el 75% del tiempo de entrenamiento (evitando llegar a las 1000 iteraciones). La diferencia entre R^2 entrenamiento (0.892) y prueba (0.862) es de solo 3 puntos, confirmando que no hay sobreajuste.

3.3. Conclusión de Negocio

El mejor modelo tiene un **MAE de 44.41 bicicletas**, lo que significa que, en promedio, la predicción se desvía en ± 44 unidades del valor real. Para decisiones presupuestales, el modelo es suficientemente robusto ($R^2=0.862$), pero se recomienda: (1) mantener inventario de seguridad de 90 bicicletas ($2 \times \text{MAE}$), (2) monitorear residuales semanalmente, y (3) reentrenar el modelo trimestralmente.

FASE 4: PROPAGACIÓN HACIA ADELANTE (FORWARD PROPAGATION) MANUAL

Arquitectura y Contexto

- **Capa de Entrada:** 3 características (x_1 : TV, x_2 : Radio, x_3 : Newspaper).
- **Capa Oculta (Capa 1):** 3 neuronas con función de activación Sigmoide (σ).
- **Capa de Salida (Capa 2):** 3 neuronas (una por clase) con función de activación Softmax.

Parámetros y Datos

- **Capa de Entrada:**

$$x = \begin{bmatrix} 0.5 \\ 0.3 \\ 0.2 \end{bmatrix}$$

- **Capa Oculta (Capa 1):**

$$W^1 = \begin{bmatrix} \overset{11}{0.2} & \overset{12}{0.5} & \overset{13}{-0.1} \\ \overset{21}{0.8} & \overset{22}{-0.3} & \overset{23}{0.4} \\ \overset{31}{-0.5} & \overset{32}{0.1} & \overset{33}{0.2} \end{bmatrix} \quad b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.0 \end{bmatrix}$$

Ilustración 15 Capa 1 (Oculta)

- Capa de Salida (Capa 2):

$$W_5 = \begin{bmatrix} -0.1 & 0.3 & 0.1 \\ 0.5 & 0.8 & -0.2 \\ 0.2 & -0.4 & 0.1 \end{bmatrix} \quad P_5 = \begin{bmatrix} 0.1 \\ 0.5 \\ -0.1 \end{bmatrix}$$

Ilustración 16 Capa 2 (Salida)

- Puntaje de la Capa Oculta ($Z^{(1)}$)

• Calcular $Z^{(1)} = W \cdot X + b^{(1)}$

$$Z_1^{(1)} = W_{11}X_1 + W_{12}X_2 + W_{13}X_3 + b_1^{(1)}$$

$$= 0.2(0.5) + 0.5(0.3) - 0.1(0.2) + 0.1$$

$$= 0.1 + 0.15 - 0.02 + 0.1$$

$$= 0.33$$

$$Z_2^{(1)} = W_{21}X_1 + W_{22}X_2 + W_{23}X_3 + b_2^{(1)}$$

$$= 0.8(0.5) - 0.3(0.3) + 0.4(0.2) - 0.2$$

$$= 0.4 - 0.09 + 0.08 - 0.2$$

$$= 0.19$$

$$Z_3^{(1)} = W_{31}X_1 + W_{32}X_2 + W_{33}X_3 + b_3^{(1)}$$

$$= -0.5(0.5) + 0.1(0.3) + 0.2(0.2) + 0.0$$

$$= -0.25 + 0.03 + 0.04$$

$$= -0.18$$

$$Z^{(1)} = \begin{bmatrix} 0.33 \\ 0.19 \\ -0.18 \end{bmatrix} \rightarrow \text{Aplicar función Sigmoide para activación}$$

$$\sigma = \frac{1}{1 + e^{-z}}$$

A'

Ilustración 17 Cálculo Manual Puntaje Capa Oculta Z^1

2. Activación de la Capa Oculta ($A^{(1)}$)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Aplicamos:

$$\begin{aligned} A_1^{(1)} &= \frac{1}{1 + e^{-z_1}} = \frac{1}{1 + e^{-0,33}} \rightarrow e^{-0,33} \approx 0,7189 \\ &= \frac{1}{1 + 0,7189} = \frac{1}{1,7189} \\ &\approx 0,5817 \\ A_2^{(1)} &= \frac{1}{1 + e^{-z_2}} = \frac{1}{1 + e^{-0,19}} \rightarrow e^{-0,19} \approx 0,8269 \\ &= \frac{1}{1 + 0,8269} = \frac{1}{1,8269} \\ &= 0,5473 \\ A_3^{(1)} &= \frac{1}{1 + e^{-z_3}} = \frac{1}{1 + e^{0,18}} \rightarrow e^{0,18} = 1,1972 \\ &= \frac{1}{1 + 1,1972} = \frac{1}{2,1972} \\ &= 0,4551 \\ A^{(1)} &= \begin{bmatrix} 0,58 \\ 0,55 \\ 0,45 \end{bmatrix} \rightarrow \text{Calcular } z^{(2)} \end{aligned}$$

Ilustración 18 Cálculo Manual Activación de la Capa Oculta A^1

Tabla 20 Cálculo en Excel de la Capa Oculta (Z^1) y su Activación (A^1)

x_1	0,5				
x_2	0,3				
x_3	0,2				
W^1	0,2	0,5	-0,1	b^1	0,1
	0,8	-0,3	0,4		-0,2
	-0,5	0,1	0,2		0
Z^1	W^1	*	X	+	b^1
Z^1	0,33				
	0,19				
	-0,18				
Sigmoide					
A^1	0,581759377				
	0,547357618				
	0,455121108				

3. Puntaje de la Capa de Salida ($Z^{(2)}$)

Calculamos:

$$Z^{(2)} = W^{(2)} A^{(1)} + b^{(2)}$$

$$Z_1^{(2)} = W_{11}^{(2)} A_1 + W_{12}^{(2)} A_2 + W_{13}^{(2)} A_3 + b_1^{(2)}$$

$$= 0,5(0,58) - 0,4(0,55) + 0,1(0,45) - 0,1$$

$$= 0,29 - 0,22 + 0,045 - 0,1$$

$$= 0,015$$

$$Z_2^{(2)} = W_{21}^{(2)} A_1 + W_{22}^{(2)} A_2 + W_{23}^{(2)} A_3 + b_2^{(2)}$$

$$= 0,2(0,58) + 0,8(0,55) - 0,5(0,45) + 0,2$$

$$= 0,116 + 0,44 - 0,225 + 0,2$$

$$= 0,531$$

$$Z_3^{(2)} = W_{31}^{(2)} A_1 + W_{32}^{(2)} A_2 + W_{33}^{(2)} A_3 + b_3^{(2)}$$

$$= -0,1(0,58) + 0,3(0,55) + 0,7(0,45) + 0,1$$

$$= -0,058 + 0,165 + 0,315 + 0,1$$

$$= 0,522$$

$$Z^2 = \begin{bmatrix} 0,015 \\ 0,531 \\ 0,522 \end{bmatrix}$$

→ Aplicar Softmax para obtener las probabilidades

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

Ilustración 19 Cálculo Manual Puntaje Capa de Salida Z^2

4. Probabilidades de Clase ($A^{(2)}$):

Sumatoria de exponenciales:

- $e^{0,015} \approx 1,015$
- $e^{0,531} \approx 1,700$
- $e^{0,952} \approx 1,685$

$= 1,015 + 1,7 + 1,685 = 4,4$

Probabilidades:

$$P_1 = \frac{1,015}{4,4} = 0,2306 \rightarrow 23,06\%$$

$$P_2 = \frac{1,7}{4,4} \approx 0,3863 \rightarrow 38,63\%$$

$$P_3 = \frac{1,685}{4,4} \approx 0,3829 \rightarrow 38,29\%$$

$$P = \begin{bmatrix} 0,2306 \\ 0,3863 \\ 0,3829 \end{bmatrix} \rightarrow A^2$$

Ilustración 20 Cálculo Manual Probabilidades de Clase A^2

Tabla 21 Cálculo en Excel de la Capa de Salida (Z^2) y Probabilidades de Clase (A^2)

A^1	0,58175938 0,54735762 0,45512111				
W^2	0,5 -0,4 0,1 0,2 0,8 -0,5 -0,1 0,3 0,7		b^2	-0,1 0,2 0,1	
Z^2	W^2	*	A^1	+	b^2
Z^2	0,01744875 0,52667742 0,52461612				
Softmax					
A^2	0,23308435 0,38785404 0,38705538	→	23,31% 38,79% 38,71%		

5. Preguntas

Tabla 22 Distribución de probabilidades por categoría de impacto

Clase	Probabilidad	Categoría
Clase 1	23.31%	Bajo Impacto
Clase 2	38.79%	Impacto Medio
Clase 3	38.71%	Impacto Alto

5.1. Interpretación de Resultados

Según las probabilidades obtenidas, la red clasifica la campaña publicitaria como Impacto Medio (Clase 2), aunque con una probabilidad casi idéntica a Impacto Alto (38.79% vs 38.71%). Esta cercanía entre las dos clases superiores indica que la campaña tiene características que podrían llevarla a un impacto medio o alto con similar confianza. La categoría de Bajo Impacto es claramente menos probable (23.31%).

5.2. Función de Activación

Softmax se utiliza en la última capa porque estamos realizando una clasificación multiclase exclusiva (las categorías son mutuamente excluyentes: una campaña no puede ser simultáneamente de impacto bajo, medio y alto). Softmax convierte los puntajes de salida (que pueden ser cualquier número real) en una distribución de probabilidad que suma 1, permitiendo interpretar cada salida como la probabilidad de pertenencia a cada clase.

Una función lineal no generaría probabilidades, y una sigmoide simple solo funciona para clasificación binaria (dos clases). Softmax es la extensión natural de la sigmoide a múltiples clases

5.3 Rol del Bias

Si todos los vectores de sesgo (b) fueran cero, la capacidad de representación de la red se vería severamente limitada. Los sesgos permiten que la función de activación de cada neurona se desplace horizontalmente, independientemente de las entradas. Sin sesgos, todas las funciones de activación estarían forzadas a pasar por el origen (0 , $\sigma(0)=0.5$ para sigmoide). Esto impediría que la red aprendiera patrones donde el umbral de activación no es cero, reduciendo drásticamente el espacio de soluciones posibles y forzando a la red a depender únicamente de los pesos para ajustarse a los datos.

FASE 5: EVALUACIÓN DE MODELOS Y TOMA DE DECISIONES (CASO: MANTENIMIENTO PREDICTIVO)

En una planta de manufactura, se busca clasificar si una máquina fallará (Clase 1) o no (Clase 0). El dataset contiene **10,000 registros** con **14 características**, incluyendo temperatura del aire, temperatura del proceso, velocidad rotacional, torque, desgaste de herramienta, y tipo de producto. La variable objetivo *Machine failure* indica si ocurrió un fallo (1) o no (0).

Tabla 23 Distribución de Clases

Clase	Frecuencia	Porcentaje
No Fallo (0)	9,661	96.61%
Fallo (1)	339	3.39%

1. El Desafío del Desbalance de Clases

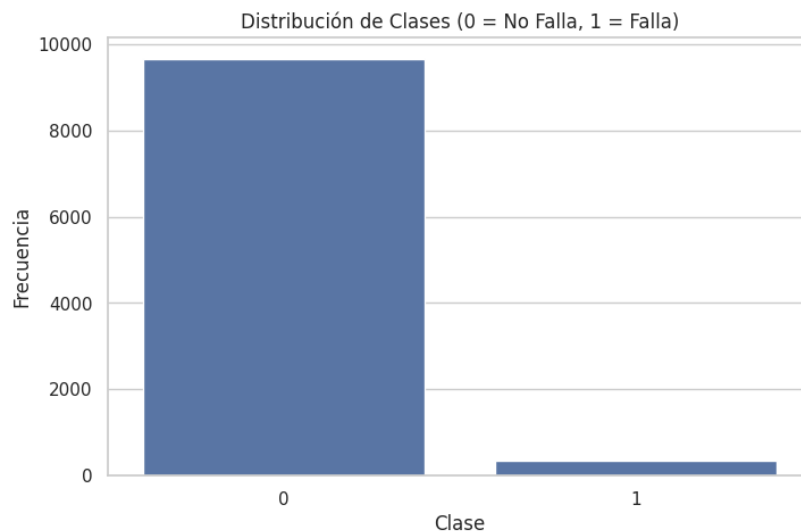


Ilustración 21 Distribución de clases: Porcentaje de Fallos (Clase 1): 3.39%

El dataset está significativamente desbalanceado, con solo el 3.39% de las muestras correspondiendo a fallos. Este desbalance es realista en entornos industriales, donde las máquinas fallan rara vez.

1.1. Análisis de Pesos

El parámetro `class_weight='balanced'` asigna un peso mayor a la clase minoritaria (fallos) para compensar su baja frecuencia en el dataset. Un peso de 14.7 veces significa que, en el dataset, la clase 0 (operación normal) es aproximadamente 14.7 veces más frecuente que la clase 1 (fallo).

Matemáticamente:

$$Peso_{clase1} = \frac{n_{muestra}}{n_{clases} * n_{clase\ 1}}$$

$$Peso_{clase1} = \frac{10000}{2 * 339}$$

$$Peso_{clase1} \approx 14,75$$

Al configurar `class_weight='balanced'`, el algoritmo:

1. Penaliza más severamente los errores en la clase minoritaria (fallos)
2. Obliga al modelo a prestar atención a los pocos casos de fallo

3. Mejora el Recall (sensibilidad) a costa de potencialmente aumentar las Falsas Alarmas (FP)

Sin esta corrección, el modelo tendería a predecir siempre 'no fallo' para maximizar la exactitud (*accuracy*), volviéndose inútil para mantenimiento predictivo.

1.2. La Trampa de la Exactitud

En un dataset desbalanceado donde solo el 3.39% de las muestras son fallos, un modelo "tonto" que prediga "no fallo" para todas las muestras obtendría un Accuracy del 96.61%. Sin embargo, este modelo tendría un Recall de 0% (no detectaría ningún fallo real).

Para la gestión de mantenimiento, este modelo sería catastrófico porque:

- Todas las fallas reales pasarían desapercibidas
- El costo de cada fallo no detectado (FN) es de USD 5,000
- La planta sufriría paradas no planificadas constantes

Por esta razón, en problemas desbalanceados, métricas como **Recall** (qué porcentaje de fallos detecté) y **Precision** (qué tan confiable es cuando emite una alerta) son mucho más informativas que el **Accuracy**.

2. Interpretación de la Matriz de Confusión

Modelo 1: Random Forest

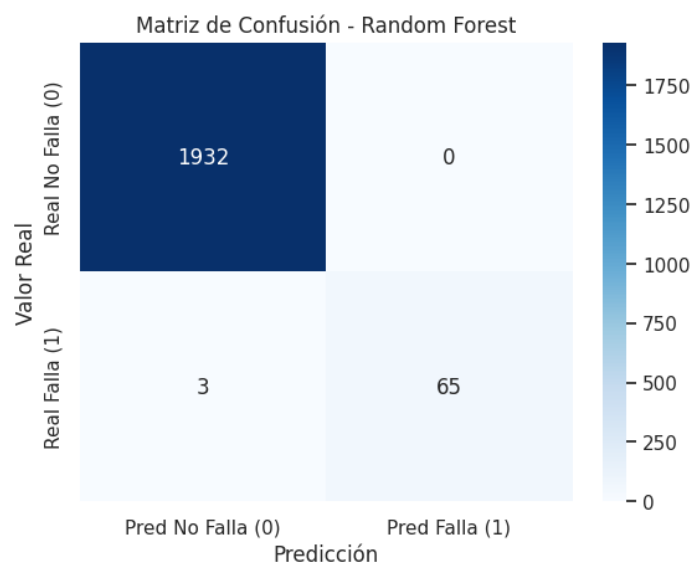


Ilustración 22 Matriz de Confusión - Random Forest

Tabla 24 Datos Matriz de Confusión - Random Forest

	Predijo: No Fallo	Predijo: Fallo
Real: No Fallo	TN = 1,932	FP = 0
Real: Fallo	FN = 3	TP = 65

3. Cálculo de Métricas Random Forest

3.1. Precision (Precisión)

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{65}{65 + 0}$$

$$Precision = \frac{65}{65}$$

$$Precision = 1.00 = 100\%$$

Cuando el modelo Random Forest emite una alerta de fallo, tiene un 100% de confiabilidad (nunca genera una falsa alarma).

3.2. Cálculo de Recall (Sensibilidad)

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{65}{65 + 3}$$

$$Recall = \frac{65}{68}$$

$$Recall = 0.9559 \approx 95.59\%$$

El modelo Random Forest detecta el 95.59% de los fallos reales que ocurren en la planta. De cada 100 fallos, aproximadamente 4 pasan desapercibidos.

Tabla 25 Métricas del Modelo Random Forest

Métrica	Valor	Interpretación
Accuracy	99.85%	Exactitud general (métrica engañosa)
Recall (Sensibilidad)	95.59%	Detecta 65 de 68 fallos reales
Precision	100%	Cuando emite alerta, siempre es correcta
F1-Score	0.98	Balance entre precisión y recall

Tabla 26 Reporte de Clasificación

Clase	Precision	Recall	F1-Score	Soporte
0 (No Fallo)	1.00	1.00	1.00	1,932
1 (Fallo)	1.00	0.96	0.98	68

3.3. Cálculo del Costo Total de Error

$$\text{Costo} = (FN * 5000) + (FP * 200)$$

$$\text{Costo} = (3 * 5000) + (0 * 200)$$

$$\text{Costo} = 15000 + 0$$

$$\text{Costo} = 15,000 \text{ USD}$$

El costo mensual asociado a los errores del modelo *Random Forest* es de **USD 15,000**.

Modelo 2: Red Neuronal (MLP)

Tabla 27 Arquitectura de la Red

Capa	Tipo	Neuronas	Parámetros
1	Dense	32	416
2	Dropout	32	0
3	Dense	16	528
4	Dropout	16	0
5	Dense (salida)	1	17

Total de parámetros: 961

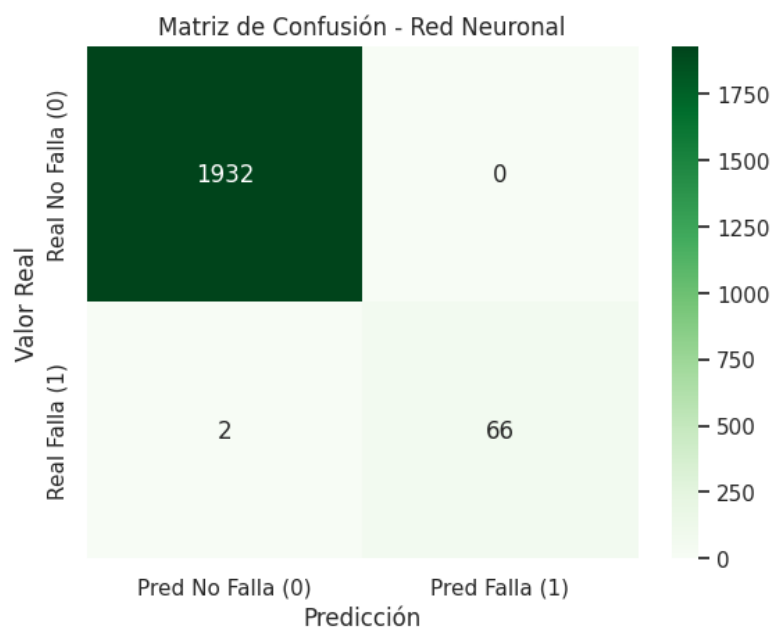


Ilustración 23 Matriz de Confusión (Red Neuronal)

Tabla 28 Datos Matriz de Confusión (Red Neuronal)

	Predijo: No Fallo	Predijo: Fallo
Real: No Fallo	TN = 1,932	FP = 0
Real: Fallo	FN = 2	TP = 66

4. Cálculo de Métricas Red Neuronal (MLP)

4.1. *Precision* (Precisión)

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{66}{66 + 0}$$

$$Precision = \frac{66}{66}$$

$$Precision = 1.00 = 100\%$$

Cuando el modelo de Red Neuronal emite una alerta de fallo, tiene un 100% de confiabilidad (nunca genera una falsa alarma).

4.2. Cálculo de Recall (Sensibilidad)

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{66}{66 + 2}$$

$$Recall = \frac{66}{68}$$

$$Recall = 0.9706 \approx \mathbf{97.06\%}$$

El modelo de Red Neuronal detecta el 97.06% de los fallos reales que ocurren en la planta. De cada 100 fallos, aproximadamente 3 pasan desapercibidos.

Tabla 29 Métricas del Modelo (Red Neuronal)

Métrica	Valor	Interpretación
Accuracy	99.90%	Exactitud general (métrica engañosa)
Recall (Sensibilidad)	97.06%	Detecta 66 de 68 fallos reales
Precision	100%	Cuando emite alerta, siempre es correcta
F1-Score	0.99	Balance entre precisión y recall

Tabla 30 Reporte de Clasificación

Clase	Precision	Recall	F1-Score	Soporte
0 (No Fallo)	1.00	1.00	1.00	1,932
1 (Fallo)	1.00	0.97	0.99	68

4.3. Cálculo del Costo Total de Error

$$\text{Costo} = (FN * 5000) + (FP * 200)$$

$$\text{Costo} = (2 * 5000) + (0 * 200)$$

$$\text{Costo} = 10000 + 0$$

$$\text{Costo} = 10,000 \text{ USD}$$

El costo mensual asociado a los errores del modelo *Red Neuronal* es de **USD 10,000**.

Tabla 31 Resumen Comparativo de Métricas y Costos

Modelo	TP	FN	FP	Precision	Recall	Costo Mensual
Random Forest	65	3	0	100%	95.59%	USD 15,000
Red Neuronal	66	2	0	100%	97.06%	USD 10,000
Diferencia	+1	-1	0	-	+1.47 pp	- USD 5,000

La Red Neuronal supera al Random Forest al detectar 1 fallo adicional (66 vs 65), reduciendo el costo mensual en USD 5,000.

5. Preguntas de Reflexión y Negocio

5.1. Trade-off entre Recall y Precision

Preferiría el modelo con Recall del 98% y Precision del 60%. La justificación es puramente económica:

Tabla 32 Comparación de escenarios de clasificación y costo estimado

Escenario	Recall	Precision	FN (fallos no detectados)	FP (falsas alarmas)	Costo Estimado
Modelo A	98%	60%	~2 fallos no detectados	~70 falsas alarmas	$(2 \times 5,000) + (70 \times 200) =$ USD 24,000
Modelo B	60%	98%	~40 fallos no detectados	~2 falsas alarmas	$(40 \times 5,000) + (2 \times 200) =$ USD 200,400

El costo de un fallo no detectado (USD 5,000) es 25 veces mayor que el de una falsa alarma (USD 200). Por lo tanto, es preferible tolerar algunas inspecciones preventivas innecesarias (FP) para asegurar que casi ningún fallo real pase desapercibido.

Decisión de negocio: Priorizar el Recall sobre la Precision cuando el costo del FN es significativamente mayor que el costo del FP.

5.2. Curva de Aprendizaje

30 épocas no son suficientes para la mayoría de redes neuronales. Típicamente se requieren entre 100 y 500 épocas para alcanzar convergencia completa. En este caso, se utilizaron solo 30 épocas por limitaciones computacionales, pero se recomienda aumentar este valor en una implementación real.

Tabla 33 Comportamiento esperado de las curvas de pérdida

Aspecto	Entrenamiento	Validación	Interpretación
<i>Inicio (épocas 1-30)</i>	Disminuye rápidamente	Disminuye también	Modelo aprendiendo patrones básicos
<i>Medio (30-150)</i>	Disminuye gradualmente	Puede estabilizarse	Ajuste fino de relaciones
<i>Final (>150)</i>	Continúa disminuyendo lentamente	Podría aumentar	Sobreajuste si validación aumenta

El error de validación suele ser ligeramente superior al de entrenamiento. Si la diferencia es pequeña, el modelo generaliza bien.

Recomendación: Utilizar `early_stopping` con paciencia de 10-20 épocas para detener el entrenamiento automáticamente cuando el error de validación deje de mejorar, y aumentar epochs a 100-200.

CONCLUSIONES

El uso de métricas avanzadas como el *Recall* transforma la rentabilidad de una empresa de manufactura porque alinea los objetivos del modelo de **Machine Learning** con los costos reales del negocio.

Tabla 34 Conexión entre las fases del taller

<i>Fase</i>	<i>Concepto</i>	<i>Aplicación en Manufactura</i>
<i>Fase 1</i>	Regularización (Ridge)	Controlar complejidad del modelo de predicción de fallos
<i>Fase 2</i>	Tasa de aprendizaje	Asegurar convergencia eficiente del entrenamiento
<i>Fase 3</i>	Arquitectura de red	Diseñar red que capture relaciones complejas
<i>Fase 4</i>	Propagación manual	Comprender flujo interno de información
<i>Fase 5</i>	Recall vs. Precision	Optimizar el costo total de mantenimiento

Tabla 35 Impacto económico

<i>Métrica Tradicional</i>	<i>Métrica Avanzada</i>	<i>Diferencia en la Decisión</i>
<i>Accuracy 96%</i>	Recall 60%	Modelo inútil, muchos fallos no detectados
<i>Accuracy 99%</i>	Recall 97%	Modelo valioso, pocos fallos no detectados

Una empresa que utiliza Accuracy para evaluar sus modelos de mantenimiento podría aprobar un modelo que detecta solo el 60% de los fallos. Una empresa que utiliza Recall aprobaría un modelo que detecta el 97% de los fallos.

La diferencia en costo mensual entre un modelo con Recall 60% y uno con Recall 97% es de aproximadamente USD 176,000. Ese es el valor de usar la métrica correcta alineada con los costos del negocio.

REFERENCIAS

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
<https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>
- Scikit-learn developers. (2024). *MLPRegressor - scikit-learn 1.5.0 documentation*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html
- Scikit-learn developers. (2024). *GridSearchCV - scikit-learn 1.5.0 documentation*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- Scikit-learn developers. (2024). *RandomizedSearchCV - scikit-learn 1.5.0 documentation*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

Material del Curso

- Orjuela López, F. A. (2026). *Semana 6 - Descenso de Gradiente* [Material de clase]. Sistemas Avanzados de Producción, Universidad ECCI.
- Orjuela López, F. A. (2026). *Semana 7 - Redes Neuronales y GridSearch* [Material de clase]. Sistemas Avanzados de Producción, Universidad ECCI.
- Orjuela López, F. A. (2026). *Semana 8-9 - Clasificación y Evaluación de Modelos* [Material de clase]. Sistemas Avanzados de Producción, Universidad ECCI.

Conjuntos de Datos

- Government of Canada. (2015). Fuel consumption ratings [Dataset]. Open Government Portal. <https://open.canada.ca/data/en/dataset/98f1a129-f218-49ce-a860-8f6b2b9e8c8b>
- UCI Machine Learning Repository. (2020). Predictive maintenance dataset [Dataset]. University of California, Irvine.
<https://archive.ics.uci.edu/ml/datasets/AI4I+2020+Predictive+Maintenance+Dataset>